Pico 2 mini OS Emulators

Pico 2 mini OS · October 27, 2026



Contents

Overview
Applesoft BASIC
  Typing programs
  The language
  Screens
  Sound
  Machine code
  DOS commands
  Files on the drive
  Memory
  Differences from a real Apple IIe
  Testing (for developers)
Apple IIe
Commodore 64
Sega Genesis

Four computers and consoles that run on the board: Applesoft BASIC, an Apple IIe, a Commodore 64 and a Sega Genesis / Mega Drive.

Overview

Four computers and consoles run on the board, each full screen on HDMI with its sound, the USB keyboard and USB gamepads. Each one's code and variables that must be fast are copied into a shared SRAM area when it starts, so they take no SRAM while they aren't running (the OS manual's Memory section). The ROMs are not included: put your own in the folders named below.

Applesoft BASIC

```
applesoft [file]
```

An Apple IIe at the Applesoft BASIC `]` prompt: its text (40 and 80 columns), lo-res and hi-res screens, and DOS 3.3 style commands (`CATALOG`, `LOAD`, `SAVE`, `RUN`, text files) working on the USB drive's current folder. With a file, it loads and runs it. `BYE` returns to the shell. No ROM is needed: it is Applesoft written again in C. The rest of this chapter has the details.

`applesoft` turns the board into an enhanced Apple IIe running Applesoft BASIC. You get its text, lo-res and hi-res screens on HDMI and its speaker on HDMI audio. DOS 3.3 style commands work on the USB drive.

```
applesoft            start at the ] prompt
applesoft GAME       load GAME (or GAME.BAS) from the drive and run it
BYE                  back to the shell
```

The aim is that programs written for a real Apple run unchanged and print exactly the same things. The interpreter keeps the program, variables and strings in an emulated 64 KB, laid out as Applesoft lays them out. It keeps Applesoft's pointers in zero page and does its arithmetic in Applesoft's 40-bit floating point, bit for bit. That means:

- numbers print with the same digits (`PRINT 1/3` gives `.333333333`, `PRINT ATN(1)*4` gives `3.14159266`), and `RND` gives the same sequence after `RND(-1)`;
- `PEEK` and `POKE` of the usual locations work: 103/104 for the program start, 105-116 for the variable and string pointers, 175/176 for the program end, 216 to 222 for ONERR, 232/233 for the shape table, and the text and graphics screens;
- `FRE(0)`, the limits on nesting FOR and GOSUB (10 FOR loops, about 25 GOSUBs) and the error messages (`?SYNTAX ERROR IN 10`) are the Apple's.

Typing programs

Type numbered lines to enter them, a bare line number to delete one, and anything else to run it at once. Keywords may be typed in lower case: they are changed to upper case except inside quotes, REM and DATA, as on the enhanced IIe. Applesoft finds keywords anywhere, spaces or not, so `SCORE` is read as `SC OR E` and `FORI=1TO10` works.

Line editing is the Apple's:

Key	Does
Left arrow, Backspace	back one character
Right arrow	copy the character under the cursor into the line
Ctrl-X	cancel the line
Esc, then I J K M or the arrows	move the cursor around the screen (escape mode); any other key ends it
Return	enter the line (the rest of the screen line is cleared)

Key	Does
Ctrl-C	stop a running program (`BREAK IN 120`); `CONT` carries on
Ctrl-S	pause output at the end of a line; any key goes on

To change a line that is on the screen, press Esc, move the cursor to the start of the line with the arrows, press any other key to leave escape mode, then copy the line with the right arrow, typing changes as you go.

The language

Everything Applesoft has is there:

- **Statements:** `CALL CLEAR COLOR= CONT DATA DEF DEL DIM DRAW END FLASH FOR GET GOSUB GOTO GR HCOLOR= HGR HGR2 HIMEM: HLIN HOME HPLOT HTAB IF IN# INPUT INVERSE LET LIST LOMEM: NEW NEXT NORMAL NOTRACE ON ONERR PLOT POKE POP PR# PRINT READ REM RESTORE RESUME RETURN ROT= RUN SCALE= SPEED= STOP TEXT TRACE VLIN VTAB WAIT XDRAW &`
- **Functions:** `ABS ASC ATN CHR$ COS EXP FN FRE INT LEFT$ LEN LOG MID$ PDL PEEK POS RIGHT$ RND SCRN( SGN SIN SPC( SQR STR$ TAB( TAN USR VAL`

Not available: `SHLOAD`, `STORE` and `RECALL`, which need a cassette. `LOAD` and `SAVE` on their own also need one; with a file name they are the DOS commands below.

Screens

Statement	Shows
`TEXT`	40 columns × 24 lines
`PR#3`	80 columns (the IIe's 80-column card); `PR#0` returns to 40
`GR`	lo-res, 40 × 40 in 16 colours, 4 text lines below
`HGR`	hi-res page 1, 280 × 160, 4 text lines below
`HGR2`	hi-res page 2, 280 × 192, full screen

The 280 × 192 picture is doubled to 560 × 384 and centred on the 640 × 480 HDMI screen. Hi-res colours come out as on an NTSC monitor: green, violet, orange and blue from alternating dots, and white where dots touch. `INVERSE` and `FLASH` text work. Programs that switch modes or pages through the soft switches (`POKE -16304,0` and the rest) are followed.

Sound

`PRINT CHR$(7)` beeps. Every touch of the speaker at $C030 (`X = PEEK(-16336)`, or machine code) clicks it, as on the Apple. The clicks are played through HDMI audio about 40 ms later, spaced exactly as they happened, so tone routines sound right.

Machine code

`CALL`, `USR` and `&` run 6502 machine code (the IIe's 65C02) at the Apple's own speed, about 1 MHz, so delay loops and tone routines behave. Programs that `POKE` code into page 3 or `BLOAD` it work. Calls into the monitor and Applesoft ROMs are carried out directly. These include

COUT, RDKEY, GETLN, HOME, the lo-res and hi-res routines, PRBYTE, WAIT and BELL; CHRGET, FRMNUM, GETBYT, GETADR, PTRGET, SYNCHR and CHKCOM for & commands; and the floating-point routines.

A call to a ROM routine that isn't covered stops the program with `?NO ROM ROUTINE AT $xxxx`.

Often-used CALLs that work: `CALL -936` (HOME), `-958` (clear to the end of the screen), `-868` (clear to the end of the line), `-198` (bell), `-3288` (tidy the stack in an ONERR handler), `54915` (clear the stack), `62450` (clear hi-res to black) and `62454` (clear hi-res to the last colour plotted).

DOS commands

Type these at the `]` prompt, or `PRINT` them after `CHR$(4)` in a program, as with DOS 3.3:

```
10 D$ = CHR$(4)
20 PRINT D$;"OPEN SCORES"
30 PRINT D$;"WRITE SCORES"
40 PRINT "ALICE": PRINT 120
50 PRINT D$;"CLOSE SCORES"
```

Command	Does
`CATALOG` (`CAT`)	list the current folder (A program, T text, B binary, D folder)
`SAVE name`	save the program as `name.BAS`
`LOAD name`	load `name` or `name.BAS`
`RUN name`	load and run it
`CHAIN name`	load and run it (variables are not kept)
`DELETE name`, `RENAME old,new`	delete or rename a file
`LOCK`, `UNLOCK`, `VERIFY name`	accepted; they only check the file exists
`BSAVE name,A$2000,L$2000`	save memory as `name.BIN`
`BLOAD name[,A$addr]`, `BRUN name[,A$addr]`	load (and run) a binary file
`EXEC name`	take keystrokes from a text file
`OPEN name[,Lreclen]`, `CLOSE [name]`	open or close a text file (up to 3)
`READ name[,Rrec][,Bbyte]`	`INPUT` and `GET` now read the file
`WRITE name[,Rrec][,Bbyte]`	`PRINT` now writes to the file
`APPEND name`	`PRINT` writes at the end of the file
`POSITION name,Rn`	skip n lines
`MON C,I,O`, `NOMON C,I,O`	show commands, file input or file output on screen
`PR#n`, `IN#n`	3: 80 columns, 0: back to normal
`BYE`	leave Applesoft

Any DOS command, or a bare `PRINT D$`, ends a READ or WRITE. DOS errors print as on DOS 3.3 (`FILE NOT FOUND`, `END OF DATA`, `SYNTAX ERROR` and the rest) and can be caught with `ONERR GOTO`, with `PEEK(222)` giving the DOS error number (5 end of data, 6 file not found, and so on).

Files on the drive

- **Programs** are saved as plain text, one line per program line, so you can read and edit them on a PC or with `edit`. `LOAD` also reads Applesoft programs in their tokenised form, such as files copied off Apple disks with CiderPress or AppleCommander. Both the DOS 3.3 file form (two length bytes first) and the bare form work.
- **Binary files** saved with `BSAVE` start with the DOS 3.3 header (load address, length). Files without one load with `,A$addr`.
- **Text files** are plain ASCII with a newline after each line.

Memory

`HIMEM` starts at 38400 ($9600), where DOS 3.3 would put it, so `FRE(0)` and the room for programs and hi-res pages are as on a 48K Apple with DOS. `HIMEM: 49152` gives another 10 KB. The program starts at $801. Hi-res page 1 is at $2000-$3FFF and page 2 at $4000-$5FFF, so a long program can run into them, as on the Apple: use `LOMEM: 24576` to put the variables above them. The 64 KB lives in the board's PSRAM.

Differences from a real Apple IIe

- Programs run far faster than on a 1 MHz Apple, apart from machine code, which keeps the Apple's speed. Empty `FOR` loops used for delays therefore end much sooner. `SPEED=` slows printing as on the Apple.
- There are no game paddles: `PDL` returns 127 and the buttons are not pressed.
- The text font is the project's own 5×7 font, not Apple's.
- Programs can't reach auxiliary memory, apart from the 80-column text screen.
- After an error inside an expression, an ONERR handler finds leftovers on the stack, as Applesoft does. `RETURN` from it then fails unless the handler does `CALL -3288` or `RESUME`.

Testing (for developers)

`src/applesoft/host` builds the interpreter on a PC (`make`). `make test` runs programs and compares what they print, and for graphics the screen memory, with output recorded from a real Apple IIe ROM. The ROM itself isn't in the project. To regenerate the expected output from your own copy of the enhanced IIe ROM (16 KB, $C000-$FFFF), use `make expected A2ROM=file`; `make fuzz A2ROM=file` checks the floating point against the ROM's own routines.

Apple IIe

```
apple2e [-d folder] [-n] [-m] [-v 0-9] [disk1 [disk2]]
```

An enhanced Apple IIe running its own ROM (in `/APPLE2E`, or `-d folder`), with a Disk II controller for 140 KB `.dsk`, `.do` and `.po` images, looked for as given and then in that folder.

- **F9** changes disks, **F10** saves changed disks and returns to the shell, **F12** is Ctrl-Reset, Alt and Win are the Open and Solid Apple keys.
- The disk runs fast (**F11** switches it; `-n` starts with it off).
- A USB joystick or gamepad is the IIe's joystick.
- A Mockingboard is in slot 4 (`-m` leaves it out); F7/F8 or `-v` set its volume (6 at first).

Commodore 64

```
c64 [-d folder] [-j 1|2] [file.prg | disk.d64 | cart.crt]
```

A PAL Commodore 64 running its own KERNAL, BASIC and character ROMs (in `/C64`, or `-d folder`).

- **Drive 8** is a `.d64` image or the folder's `.prg` files; `LOAD` and `SAVE` work on both. A file given is started once BASIC is up.
- **Cartridges** (`.crt`): normal, Ultimax, Ocean, System 3, Magic Desk and EasyFlash, from the command line or the F9 menu (F3 there takes one out).
- **F9** is the menu (disks, programs, cartridges, the joystick port; it remembers its folder), **F10** returns to the shell, **F11** turns the C64 off and on, **F12** is RESTORE. In the menu, F4 makes drive 8 a true 1541 (its ROM goes in the folder too).
- Keys: Esc is RUN/STOP, Tab is CTRL, Ctrl or Win is the C= key.
- **Joysticks:** USB gamepads (the first in port 2, a second in port 1), or the keypad (8 4 6 2, fire 0 or 5) as the first; keypad + and - set the volume.

Sega Genesis

```
genesis [-d folder] [-f] [rom.bin]
```

A Sega Genesis / Mega Drive. Without a ROM it shows the games in `/GENESIS` (or `-d folder`) to pick one: Return or Start plays, Esc leaves. A ROM is looked for as given, then there. The list remembers where it was.

- Arrows; Z, X and C are A, B and C; Return is Start; or a USB gamepad (a second one is pad 2).
- **F9** in a game goes back to the list, **F11** resets, **F10** returns to the shell.
- `-f` gives full-quality FM sound (busy games may stutter); `-b n` runs a benchmark of n frames.